

Especificación Arquitectónica: Plataforma de Gobernanza Documental Inteligente

Este documento define la estructura técnica, los patrones de diseño y las tecnologías a utilizar en el desarrollo de la plataforma, dividida en dos entornos principales: **Nube (Web SaaS)** y **Borde (Escritorio Edge)**.

1. Arquitectura Global (Macro-Arquitectura)

El sistema utiliza una **Arquitectura Cliente-Servidor Distribuida con Edge Computing**.

- **Edge Computing (Cliente Local):** Absorbe la carga computacional pesada (IA, criptografía) para proteger la privacidad ("Zero Trust") y funcionar en modo Offline.
- **Central Cloud (SaaS):** Centraliza la administración (RBAC), reglas de negocio globales y consolida la auditoría (Trazabilidad Forense).

2. Entorno en la Nube (construcción-web)

Este entorno gestiona la configuración de políticas y visualización de reportes.

2.1. Patrones y Arquitectura

- **Arquitectura:** Monolito Full-Stack Modular basado en capas.
- **Patrón de Interfaz/Backend: BFF (Backend For Frontend)** utilizando las API Routes de Next.js.
- **Patrón de Acceso a Datos: Repository Pattern** (implementado implícitamente a través de Prisma ORM) y **Active Record** a nivel de modelo.

2.2. Tecnologías Core

- **Framework Principal:** Next.js (con App Router) - Permite SSR (Server-Side Rendering) y manejo de API.
- **Lenguaje:** TypeScript (Estricto) para evitar errores en tiempo de ejecución.
- **UI / Estilos:** React.js, Tailwind CSS, shadcn/ui (recomendado para componentes rápidos).
- **Capa de Datos:** Prisma ORM.
- **Base de Datos Principal:** MySQL.
- **Autenticación:** NextAuth.js (o JSON Web Tokens - JWT para los clientes de escritorio).

2.3. Capas del Sistema (Nube)

1. **Capa de Presentación (Frontend):** Componentes React que el administrador ve e interactúa.
2. **Capa de Aplicación / Controladores (API Routes):** Funciones Serverless de Next.js que reciben las peticiones de la web y del escritorio.

3. Capa de Acceso a Datos: Modelos de Prisma que mapean los objetos a tablas en MySQL.

2.4. Estructura de Carpetas (construccion-web/)

└─ construccion-web/	
└─┬─ prisma/	# Definición de la Base de Datos
└─└─ schema.prisma	# Modelos: User, Role, Workspace, AuditLog, Confi
└─┬─ public/	# Recursos estáticos (Logos, favicons)
└─┬─ src/	
└─└─┬─ app/	# [Next.js App Router]
└─└─└─┬─ api/	# 🍷 CAPA BACKEND (BFF)
└─└─└─└─┬─ auth/	# Login/Autenticación
└─└─└─└─└─┬─ sync/	# Endpoint para recibir escaneos del escritorio
└─└─└─└─└─└─┬─ policies/	# Endpoints para obtener/crear reglas
└─└─└─└─└─└─└─┬─ dashboard/	# 🍷 CAPA FRONTEND
└─└─└─└─└─└─└─└─┬─ users/	# Gestión de empleados (Soft Delete)
└─└─└─└─└─└─└─└─└─┬─ logs/	# Visor de Auditoría Forense
└─└─└─└─└─└─└─└─└─└─┬─ page.tsx	# Panel principal
└─└─└─└─└─└─└─└─└─└─└─┬─ layout.tsx	# Wrapper principal con contexto de autenticación
└─└─└─└─└─└─└─└─└─└─└─└─┬─ page.tsx	# Landing page / Login de Administrador
└─└─└─└─└─└─└─└─└─└─└─└─└─┬─ components/	# Componentes React (Botones, Tablas, Modales)
└─└─└─└─└─└─└─└─└─└─└─└─└─└─┬─ lib/	# Utilidades globales
└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─┬─ prisma.ts	# Instancia Singleton del ORM
└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─┬─ utils.ts	# Funciones de ayuda (formato de fechas, etc.)
└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─┬─ types/	# Interfaces y tipos globales de TypeScript
└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─┬─ .env	# Variables: DATABASE_URL, JWT_SECRET, etc.
└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─└─┬─ next.config.js	# Config. del framework
└─┬─ tailwind.config.js	# Config. de utilidades CSS
└─┬─ package.json	# Dependencias de Node

3. Entorno de Escritorio (construccion-escritorio)

Este es el cliente instalado en la máquina del usuario final. Actúa como un "Fat Client" inteligente.

3.1. Patrones y Arquitectura

- **Arquitectura Central: Arquitectura Limpia (Clean Architecture)** en el núcleo de Python, separando las interfaces (UI/Hardware) de la lógica pura (IA/Criptografía).
- **Patrón de Ejecución: Arquitectura Orientada a Eventos (EDA)** y patrón **Productor-Consumidor** para manejar el procesamiento en segundo plano (Workers) y evitar congelar la UI.
- **Comportamiento: Offline-First.** Toda operación nace local y luego se sincroniza.

3.2. Tecnologías Core

- **Lenguaje Backend Local:** Python 3.10+.
- **Puente Interfaz-Backend:** Eel (empaqueta UI web + Python en un .exe).
- **Interfaz (UI Local):** HTML5, Vanilla JavaScript, Tailwind CSS (para diseño consistente con la web).

- **IA / Visión / OCR:** PyTesseract (OCR), OpenCV (Computer Vision / Template Matching).
- **Seguridad:** PyCryptodome (AES-256 en modo GCM/CBC).
- **Base de Datos Local:** SQLite.
- **Hardware Bridge:** Librerías `twain` o `pywia` para comunicación nativa en Windows.

3.3. Capas del Sistema (Escritorio)

1. **Capa de Presentación (UI Eel):** Renderiza el progreso, permite correcciones manuales y seleccionar espacios.
2. **Capa de Control / Eventos:** Intercepta eventos de JS y los lanza en colas de Python.
3. **Capa de Dominio / Servicios:** Motores de IA, clasificación y cifrado.
4. **Capa de Infraestructura Local:** Drivers TWAIN, SQLite, módulo de red (HTTP Requests a Next.js y nube).

3.4. Estructura de Carpetas (`construccion-escritorio/`)

```

└─ construccion-escritorio/
  └─ web/
    └─ css/
    └─ js/
      └─ app.js
      └─ eel_bridge.js
    └─ assets/
    └─ index.html
    └─ correction.html
  └─ src/
    └─ core/
      └─ ocr_engine.py
      └─ vision_engine.py
      └─ crypto_engine.py
    └─ hardware/
      └─ scanner.py
    └─ database/
      └─ sqlite_manager.py
    └─ sync/
      └─ cloud_uploader.py
      └─ saas_api.py
    └─ controller.py
  └─ data/
    └─ local_queue.db
    └─ temp_workspace/
  └─ main.py
  └─ requirements.txt
  └─ build.spec
  
```

👉 CAPA DE PRESENTACIÓN (Frontend Local)

Estilos (Tailwind compilado)

Lógica de la interfaz

Manejo del DOM

Promesas de comunicación Eel -> Python

Imágenes locales, íconos del sistema

Pantalla principal (Selección de Espacio)

Interfaz para corregir PDFs rechazados por la I

👉 CAPA DE APLICACIÓN Y DOMINIO (Python)

Lógica pura de negocio (Motores)

Tesseract: Extracción de texto y nomenclaturas

OpenCV: Búsqueda de sellos de "Secreto"

Encriptación AES-256 y jerarquía de anulación

Capa de dispositivos

Wrapper TWAIN/WIA y manejo de atascos

Capa de resiliencia

Operaciones CRUD sobre la bandeja de pendientes

Sincronización con SaaS y Nube

Comunicación con GDrive/Dropbox

Cliente HTTP para enviar Logs a Next.js

Funciones expuestas a Javascript (@eel.expose)

Almacenamiento Dinámico (Ignorado en Git)

Base de datos SQLite

Directorio temporal de PDFs durante sesión

Punto de entrada. Levanta Eel y los hilos (Work

Lista de librerías Python

Archivo de configuración para PyInstaller (crec

4. Patrones de Diseño de Software Recomendados

Para mantener la robustez y escalabilidad a medida que el proyecto crezca, se deben aplicar los siguientes patrones:

1. Patrón Strategy (Estrategia):

- *Uso:* En `construccion-escritorio/src/sync/`.
- *Por qué:* Te permite tener diferentes "estrategias de subida" (ej. `GoogleDriveStrategy`, `DropboxStrategy`). La lógica principal de sincronización no cambia, solo intercambias la estrategia según lo configurado por el Administrador.

2. Patrón Observer / PubSub (Observador):

- *Uso:* Comunicación entre Python y la interfaz web local de Eel.
- *Por qué:* Mientras el `ocr_engine.py` procesa páginas, emite eventos (`"page_processed"`, `"encryption_done"`). La UI de JavaScript "observa" estos eventos y actualiza la barra de progreso sin bloquearse.

3. Patrón Singleton:

- *Uso:* Conexiones a bases de datos (`Prisma` en la web, `sqlite_manager.py` en el escritorio).
- *Por qué:* Evita múltiples instancias abriendo conexiones concurrentes que pueden bloquear el archivo SQLite o saturar MySQL.

4. Patrón Unit of Work (Unidad de Trabajo):

- *Uso:* Al confirmar un escaneo local y sincronizar.
- *Por qué:* Permite asegurar transacciones atómicas. Si la subida a GDrive es exitosa, pero el reporte del Log a Next.js falla, la unidad de trabajo hace un *rollback* (mantiene el archivo en SQLite) para reintentar después, garantizando la consistencia de los datos.

Este documento actúa como la guía maestra de construcción. Cualquier desviación arquitectónica debe ser consultada con este plano.